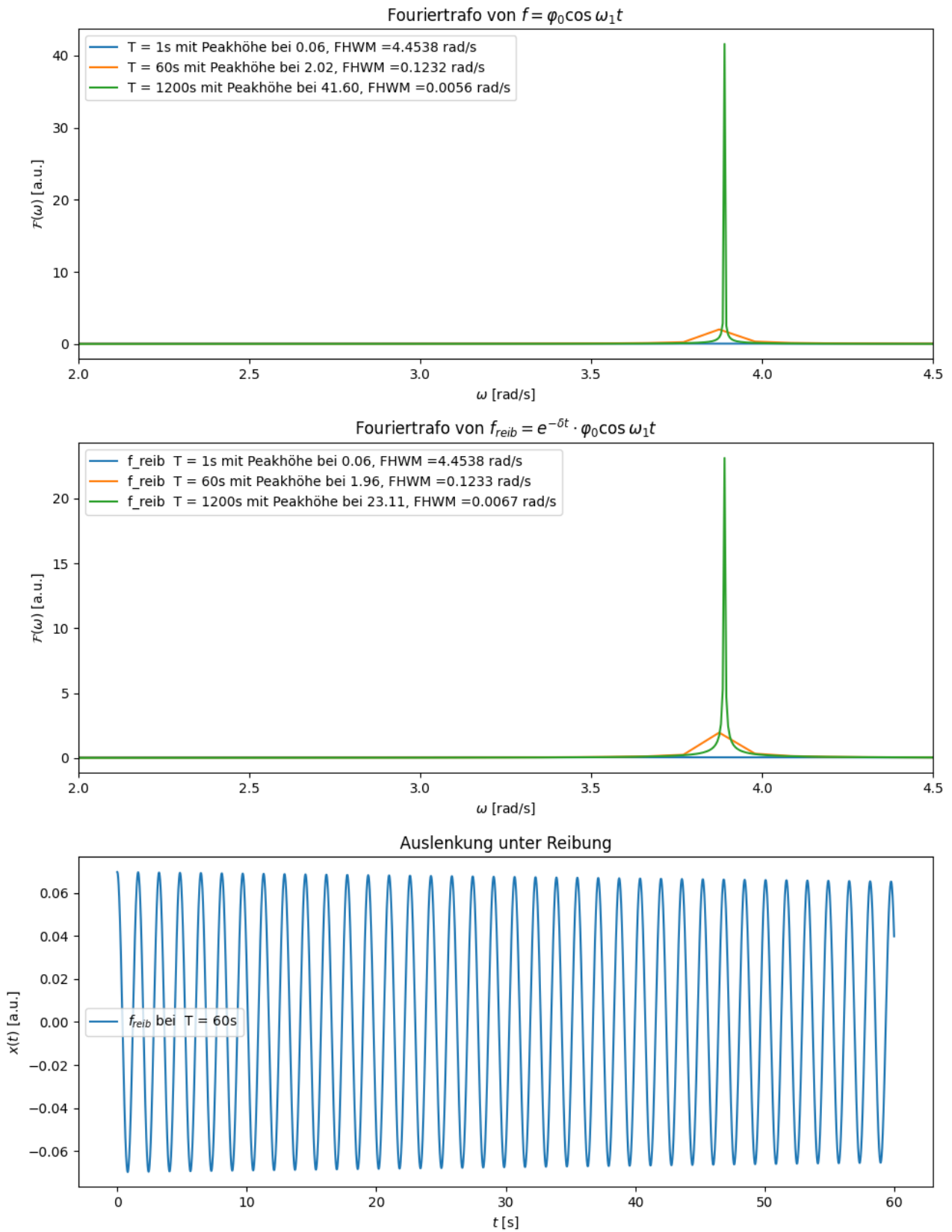


Die Auswirkung der Reibungseffekte auf die FFT konnte ich mir anhand der mathematischen Formel nicht vorstellen. Deswegen habe ich jetzt die FFT der Funktion $f_{reib} = e^{-\delta t} \cdot \varphi_0 \cos \omega_1 t$ geplottet. δ habe ich rudimentär aus zwei Messpunkten der $\text{tx}(\tau)$ Datei berechnet.



Man sieht, die Amplitude der Peaks sinkt, die FWHM wird jedoch nur bei nicht signifikanten Stellen minimal größer. Auch die x-Stelle der Peaks, also die Frequenz ω_1 bleibt am exakt gleichen Ort. Reibungseffekte können also komplett vernachlässigt werden.

Nun die Frage: Warum schreibst du aber in der Diskussionshilfe: „[mit dem Wissen, dass] Reibung einem/einer einen Strich durch die Rechnung machen wird[...]“. Das impliziert, dass Reibung die Messergebnisse signifikant verfälscht hat, was aber nach den Plots nicht so ist.

```

F_shift_coll, omega_shift_coll = [], [] #Listen, in denen FFT Größen aus der
↳For-Schleife (Iterationen für jede Zeit in T) gespeichert werden
FHWm=[]
F_shift_coll2, omega_shift_coll2 = [], [] #Listen, in denen FFT Größen aus der
↳For-Schleife (Iterationen für jede Zeit in T) gespeichert werden
FHWm2=[]
peakamplitude1=[]
peakamplitude2=[]

for index,i in enumerate(T):
    T = i #Aktuelle Messzeit (i) wird als Variable T gespeichert
    N = int(T/0.002) #Anzahl insgesamt gemessener Zeitschritte (2ms ist laut
↳Skript die Abtast/Aufnahmerate)
    x = np.linspace(0, T, N, endpoint=False)
    f = phi * np.cos(a * x) #Funktion für symmetrische/asymmetrische Schwingung
    freib=np.exp(-0.0011*x)*f
    dx = x[1] - x[0]
    F = np.fft.fft(f) * dx #Fouriertransformierte der Zeitfunktion (Schwingung).
↳ Das dx ist notwendig, um von der diskreten Form in die kontinuierliche zu
↳gehen
    omega = 2 * np.pi * np.fft.fftfreq(N, d=dx) # erzeugt die x-Achsenwerte
    F_shift = np.fft.fftshift(F) #Sortiert die Frequenzen von klein (minus)
↳nach groß, da sie in F komisch sortiert sind
    omega_shift = np.fft.fftshift(omega)
    omega_shift_coll.append(omega_shift)
    reellefreq = np.where(omega_shift > 0, np.abs(F_shift), 0) # Macht die
↳linke Hälfte platt
    F_shift_coll.append(reellefreq) #Fügt Ergebnisse in die Sammeliste ein
    peaks, xindex = find_peaks(reellefreq) # spuckt Array aus Indexen
↳(Stellen) des w-Arrays(x-Achse) aus, an denen ein Peak ist
    main_peak = peaks[np.argmax(reellefreq[peaks])] # mit
↳reellefreq[peaks] werden alle Amplituden der Indexstellen, die in [peaks]
↳gespeichert sind, herausgeholt und sozusagen als phantom array gespeichert.
↳davon wird mit np.argmax der Index des maximalsten Wertes aus diesem phantom
↳array genommen. Und mit peaks[Index_maximaler_Wert] der mainpeak extrahiert.
    print(omega_shift[main_peak])
    results = peak_widths(reellefreq, [main_peak], rel_height=0.5)
    FHWm.append(results[0][0]*2*np.pi*1/T)
    peakamplitude1.append(reellefreq[main_peak])

    F2 = np.fft.fft(freib) * dx #Fouriertransformierte der Zeitfunktion
↳(Schwingung). Das dx ist notwendig, um von der diskreten Form in die
↳kontinuierliche zu gehen
    omega2 = 2 * np.pi * np.fft.fftfreq(N, d=dx)
    F_shift2 = np.fft.fftshift(F2) #Sortiert die Frequenzen von klein (minus)
↳nach groß

```

```

omega_shift2 = np.fft.fftshift(omega2)
F_shift_coll2.append(F_shift2) #Fügt Ergebnisse in die Sammeliste ein
omega_shift_coll2.append(omega_shift2)
reellefreq2=np.abs(F_shift2)
peaks2, _ = find_peaks(reellefreq2)
main_peak2 = peaks2[np.argmax(reellefreq2[peaks2])]
print(+omega_shift2[main_peak2])
results2 = peak_widths(reellefreq2, [main_peak2], rel_height=0.5)
FHWM2.append(results2[0][0]*2*np.pi*1/T)
peakamplitude2.append(reellefreq2[main_peak2])

if index==1:
    ax3.plot(x, freib, label = r'$f_{reib}$ bei ' + labels[index]) ␣
↪#Plottet jedes Amplitudenspektrum (Amplitude der Fouriertransformierten) für
↪jede Messzeit T in einen Plot
    # ax2.plot(x,f, label = labels[j])

'''
Erstellung der Plots
'''
ax1.set_xlabel(r'$\omega$ [rad/s]')
ax1.set_ylabel(r'$\mathcal{F}(\omega)$ [a.u.]')
ax1.set_xlim(-4.5,4.5) #Interessanter Frequenzbereich (je nach Präferenz
↪variabel)
ax1.set_title(r'Fouriertrafo von $f=\varphi_0\cos \omega_1t$')
for j in np.arange(0,len(F_shift_coll)):
    ax1.plot(omega_shift_coll[j], F_shift_coll[j], label = labels[j]+ f" mit
    ↪Peakhöhe bei {peakamplitude1[j]:.2f}, FHWM={FHWM[j]:.4f} rad/s") #Plottet
↪jedes Amplitudenspektrum (Amplitude der Fouriertransformierten) für jede
↪Messzeit T in einen Plot
ax1.legend(loc = 'best')

ax2.set_xlabel(r'$\omega$ [rad/s]')
ax2.set_ylabel(r'$\mathcal{F}(\omega)$ [a.u.]')
ax2.set_xlim(-4.5,4.5) #Interessanter Frequenzbereich (je nach Präferenz
↪variabel)
ax2.set_title(r'Fouriertrafo von $f_{reib}=e^{-\delta t} \cdot \varphi_0\cos
    \omega_1t$')
for j in np.arange(0,len(F_shift_coll)):
    ax2.plot(omega_shift_coll2[j], np.abs(F_shift_coll2[j]), label = r'f_reib
    ↪' + labels[j]+ f" mit Peakhöhe bei {peakamplitude2[j]:.2f}, FHWM={FHWM2[j]:.
↪4f} rad/s") #Plottet jedes Amplitudenspektrum (Amplitude der
↪Fouriertransformierten) für jede Messzeit T in einen Plot
ax2.legend(loc = 'best')

ax3.set_xlabel(r'$t$ [s]')

```

```

ax3.set_ylabel(r'$x(t)$ [a.u.]')
# ax2.set_xlim(2,4.5) #Interessanter Frequenzbereich (je nach Präferenz,
↪variabel)
ax3.set_title(r'Auslenkung unter Reibung')

ax3.legend(loc = 'best')

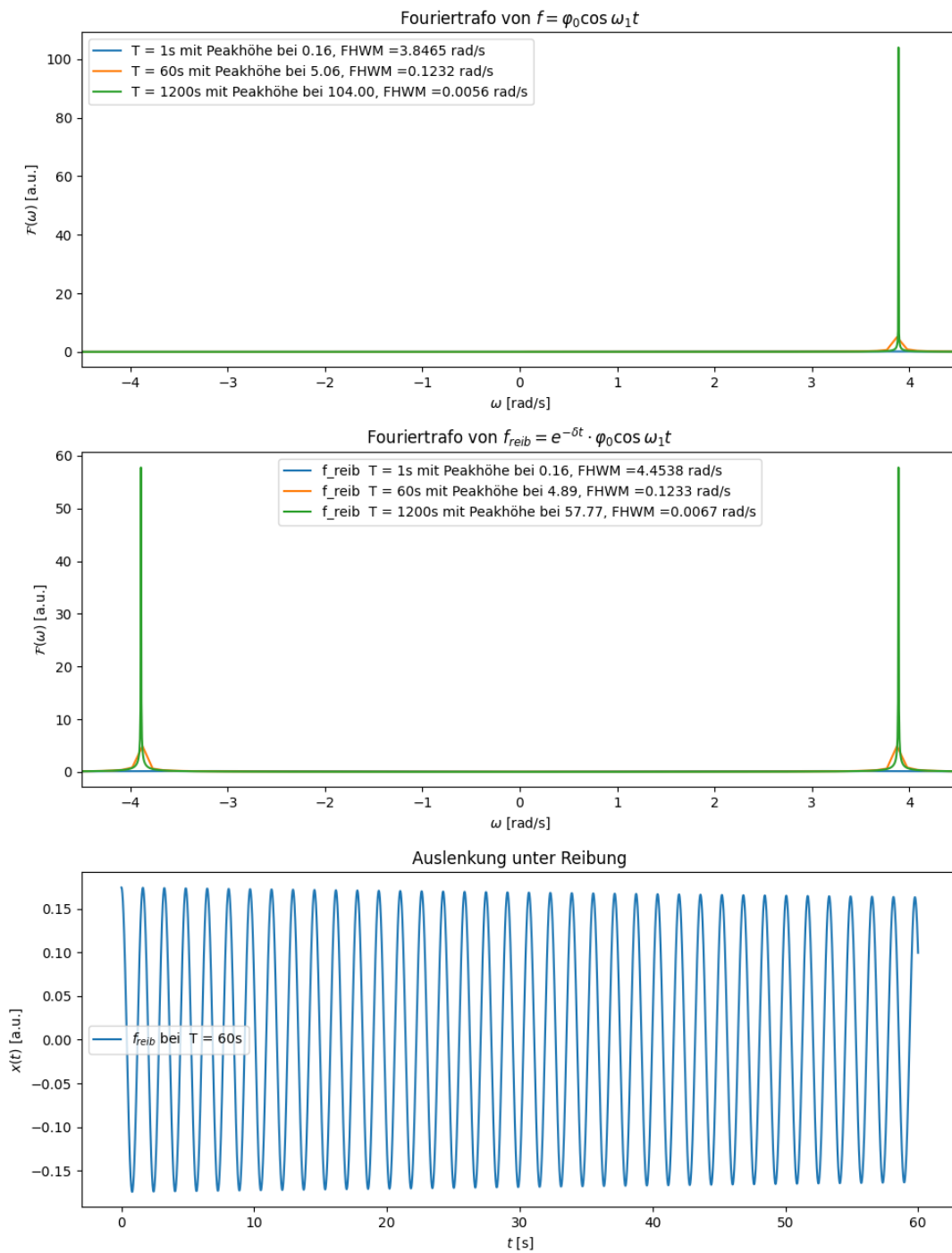
plt.tight_layout()
plt.show()
# plt.savefig(Ausgabe_path/"Fouriertrafos.png",format="png")

```

```

3.141592653589793
-3.141592653589793
3.8746309394274117
-3.8746309394274117
3.890338902695361
-3.890338902695361

```



[11]: `# i looked up in the tx(t) file, that $x(t=0)=92$, $x(t=58.13)=86$
 # so we can calculate delta out of these two points, we only need $x(t=0)$ to $\square$
 $\hookrightarrow$ calculate the scale of the a.u. units compared to the scale of $\cos(\dots)$`

```
scale=92/(phi*np.cos(a*0))
delta=-np.log(86/(scale*phi*np.cos(a*58.13)))/58.13
print(f"delta={delta}")
x_58=np.exp(-delta*58.13)*scale*phi*np.cos(a*58.13)
print(x_58)
```

delta=0.0011192308478834433

86.0

LETS GO, passt also, dann haben sich die .tx(t) dateien doch gelohnt hah